

CHAPTER FOUR NORMALIZATIONS

Functional Dependency and Data Anomalies

A functional dependency is a constraint between two sets of attributes from the database. Suppose that our relational database schema has n attributes $A_1, A_2, \dots, A_n$; let us think of the whole database as being described by a single universal relation schema $R = \{A_1, A_2, \dots, A_n\}$. We do not imply that we will actually store the database as a single universal table; we use this concept only in developing the formal theory of data dependencies.

*Definition:- A functional dependency, denoted by $X \rightarrow Y$, between two sets of attributes X and Y that are subsets of R specifies a constraint on the possible tuples that can form a relation state r of R . The constraint is that, for any two tuples **t_1 and t_2** in r that have $t_1[X] = t_2[X]$, they must also have $t_1[Y] = t_2[Y]$. This means that the values of the Y component of a tuple in r depend on, or are determined by, the values of the X component; alternatively, the values of the X component of a tuple uniquely (or functionally) determine the values of the Y component. We also say that there is a functional dependency from X to Y , or that Y is functionally dependent on X .*

The abbreviation for functional dependency is FD or f.d. The set of attributes X is called the left-hand side of the FD, and Y is called the right-hand side. The functional dependency is a relationship that exists between two attributes. It typically exists between the primary key and non-key attribute within a table. A functional dependency is a property of the semantics or meaning of the attributes. The database designers will use their understanding of the semantics of the attributes of R —that is, how they relate to one another—to specify the functional dependencies that should hold on all relation states (extensions) r of R .

Trivial functional dependency

$A \rightarrow B$ has trivial functional dependency if B is a subset of A .

The following dependencies are also trivial like: $A \rightarrow A, B \rightarrow B$

Non-trivial functional dependency

$A \rightarrow B$ has a non-trivial functional dependency if B is not a subset of A .

When A intersection B is NULL, then $A \rightarrow B$ is called as complete non-trivial.

Functional dependency and data anomalies are the important database design considerations. Data anomalies are simply inconsistency of data arising from manipulations like insert update and delete operations.

Normalization

- ✓ Normalization is the step by step process of organizing the data in the database.
- ✓ Normalization is used to minimize the redundancy from a relation or set of relations. It is also used to eliminate the undesirable characteristics like Insertion, Update and Deletion Anomalies.
- ✓ Normalization divides the larger table into the smaller table and links them using relationship.

The Normalization process that is used to reduce redundancy from the database table is known as **normal form**. The normal form includes:-

- ❖ First Normal form (1NF)
- ❖ Second Normal form (2NF)

Chapter Four Normalizations

- ❖ Third Normal form (3NF)
- ❖ Boyce Codd Normal form (BCNF)
- ❖ Fourth Normal form (4NF)
- ❖ Fifth Normal form (5NF)
- ❖ Sixth Normal form (6NF)

Types of Normal Forms	Description
Normal Form	
1NF	A relation is in 1NF if it contains an atomic value.
2NF	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.
3NF	A relation will be in 3NF if it is in 2NF and no transition dependency exists.
4NF	A relation will be in 4NF if it is in Boyce Codd normal form and has no multi-valued dependency.
5NF	A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.

First Normal Form (1NF)

- A relation/table is said to be 1NF, if it contains an atomic value.
 - It states that an attribute of a table cannot hold multiple values. It must hold only single-valued attribute.
 - First normal form disallows the multi-valued attribute, composite attribute, and their combinations.

Example1: Relation EMPLOYEE is not in 1NF because of multi-valued attribute EMP_PHONE.

EMPLOYEE table :

EMP_ID	FNAME	LNAME	Salary	Annual Salary	Dep_Name	Dep_Head
14	John	Dave	2000	24000	ACCOUNTNG	Bob
20	Harry	Stive	3000	36000	COMPUTE	Alice
12	Sam	Smith	2500	30000	ACCOUNTNG	Bob

The decomposition of employee name the EMPLOYEE table into 1NF has been shown above:

Second Normal Form (2NF)

- A relation/table is said to be 2NF, if it
 - is in 1NF.
 - has all non-key attributes are fully functional dependent on the primary key

Example 2: the above employee table in *Example 1* is in 1NF. But it has column group **Dep_Name and Dep_head** which are none key attribute and do not depend on the primary key. This means the change on this column group have no direct relation with the change on the primary key. This scenario leads to redundancy of values for the columns. To avoid the problem of redundancy the table/relation has to be normalized to 2NF as in below.

Chapter Four Normalizations

Dep_Id	Dep_Name	Dep_Head
01	ACCOUNTNG	Bob
02	COMPUTE	Alice

EMP_ID	FNAME	LNAME	Salary	Annual Salary	Dep_Id
14	John	Dave	2000	24000	01
20	Harry	Stive	3000	36000	02
12	Sam	Smith	2500	30000	01

To convert the given table into 2NF, we decompose it into two tables:

Third Normal Form (3NF)

A relation/table will be in 3NF if it is in 2NF and does not contain any transitive functional dependency.

- ✓ 3NF is used to reduce the data duplication.
- ✓ It is also used to achieve the data integrity.
- ✓ If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.
- ✓ A relation is in third normal form if it holds at least one of the following conditions for every non-trivial function dependency $X \rightarrow Y$.

Example 3: The above employee table in **Example 2** is in 2NF. But it has column group **salary and Annual_salary** which are none key attribute and are functionally dependent as **Annual_salary = (12 * salary)**. To avoid this, the calculated “**Annual_salary**” column has to be dropped as in below.

Dep_Id	Dep_Name	Dep_Head
01	ACCOUNTNG	Bob
02	COMPUTE	Alice

EMP_ID	FNAME	LNAME	Salary	Dep_Id
14	John	Dave	2000	01
20	Harry	Stive	3000	02
12	Sam	Smith	2500	01

To convert the given table into 3NF, we dropped the annual salary column.

Boyce Codd normal form (BCNF)

- ✓ BCNF is the advance version of 3NF. It is stricter than 3NF.
- ✓ A table is in BCNF if every functional dependency $X \rightarrow Y$, X is the super key of the table.
- ✓ For BCNF, the table should be in 3NF, and for every FD, LHS is super key.

. Fourth normal form (4NF)

- ✓ A relation will be in 4NF if it is in Boyce Codd normal form and has no multi-valued dependency.
- ✓ For a dependency $A \twoheadrightarrow B$, if for a single value of A , multiple values of B exists, then the relation will be a multi-valued dependency.

Fifth normal form (5NF)

- ✓ A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.
- ✓ 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.
- ✓ 5NF is also known as Project-join normal form (PJ/NF).